

Module-5 Turing Machines.

Syllabus.

1. Introduction to Turing Machines.
2. Construction of Turing Machines.
 - Problems with Transition Table
 - Types of Turing Machine.
 - Halting Problem of TM.
 - Undecidability: A language That is Not Recursively Enumerable.

Introduction to Turing Machine.

Turing Machine Model.

The Turing Machine Model is shown below. It is a finite automaton connected to read-write head with the following Components:

- (i) Tape
- (ii) Read-Write head
- (iii) Control Unit

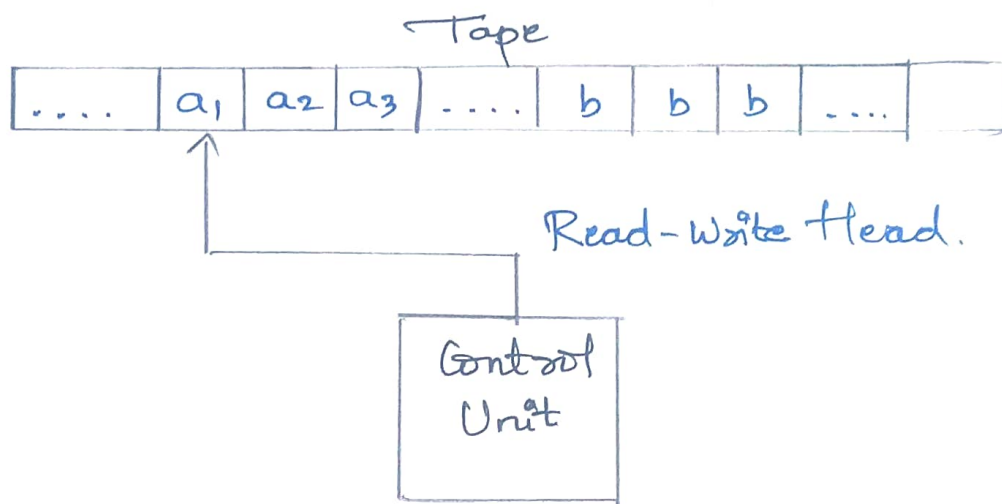


figure: Turing Machine Model.

Tape: It is a temporary storage and is divided into cells. Each cell can store the information of only one symbol. The string to be scanned will be stored from the left most position on the tape. The string to be scanned should end with blanks.

Read-Write Head: The read-write head can read a symbol from where it is pointing to and it can write into the tape to where the read-write head points to.

Control Unit: The reading from the tape or writing into the tape is determined by the Control Unit. The different moves performed by the machine depends on the current scanned symbol and the current state. The read-write head can move either towards left or right i.e., movement can be on both

the directions. The various moves performed by the machine are:

1. Change of state from one state to another state.
2. The symbol pointing to by the read-write head can be replaced by another symbol.
3. The read-write head may move either towards left or towards right.

Definition of Turing Machine:

A Turing Machine M is a 7-tuple, namely.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F).$$

where,

Q = It is finite non-empty set of states.

Σ = It is finite non-empty set of input alphabets.

Γ = It is a set of tape symbols.

δ = It is a transition function from $Q \times \Gamma$ to $Q \times \Gamma \times \{L, R\}$.

q_0 = It is the start state, $q_0 \in Q$.

B = It is a special symbol indicating blank character.

$F \subseteq Q$, = It is set of final states.

Representation of Turing Machines.

(i) Transition Table

(ii) Instantaneous Description.

(iii) Transition Diagram.

(i) Transition Table.

Definition: The transition of TM is represented using table is called transition table of a Turing machine.

Consider, for example, a Turing machine with five states $q_0, q_1, \dots, q_4$ where q_0 is the start state and q_4 is the final state.

δ States	Tape Symbol (Σ)				
	a	b	x	y	B.
q_0	(q_1, x, R)	—	—	(q_3, y, R)	—
q_1	(q_1, a, R)	(q_2, y, L)	—	(q_4, y, R)	—
q_2	(q_2, a, L)	—	(q_0, x, R)	(q_2, y, L)	—
q_3	—	—	—	(q_3, y, R)	(q_4, B, R)
q_4	—	—	—	—	—

q_0 is start state, there are corresponding entry for the symbol in Σ . for each of the states from Q . The symbols a, b, x, y & B are denoted by Σ & $\Sigma \subseteq \Sigma'$. The Symbol B indicates a blank character and usually the string ends with infinite number of B 's i.e. Blank character.

The undefined entries in the table indicate that there are no-transitions defined or there can be transition to dead state. When there is a transition to the dead state. the machine halts and the input string is rejected by the machine. In general, δ can be defined as follows.

$$\delta: Q \times \Sigma \rightarrow (Q \times \Sigma \times \{L, R\}).$$

(ii) Instantaneous Description.

The ID's of a Turing machine is defined in terms of the entire input string and the current state.

Definition:

An ID of TM is defined as a string $\alpha q \beta$ where.

- * q is the current state of TM.
- * given string is divided into two substring α & β such that given string is obtained by concatenating α & β . The read-write head points to the first character of the substring β .

* The initial ID is denoted by $q\alpha\beta$ where q is the start state and the read-write head points to the first symbol of α from left.

* The final ID is denoted by the string $\alpha\beta q\beta$ where $q \in F$ is the final state and the read-write head points to the blank character denoted by B .

(iii) Transition Diagram.

The transition diagram for a TM

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

is defined as a graph with circles, arrows, arcs with labels and two concentric circles

where,

* Each state is called as vertex, denoted using a circle or two circles.

* Each edge is labelled with $(a, X, L/R)$ where a is the current input symbol read from a tape which has to be replaced by X on the tape moving the read/write head to left or right.

* The transition from one state to another state is indicated by a directed edge. Let $\delta(q_i, a) = q_j$. This indicates that there is a directed edge from q_i to q_j & the edge is labelled a .

* The start state is a state which has an arrow not originating from any node and entering into the state. This is labelled with " $\rightarrow$ ".

* The final states or accepting states which are in F are represented by double circles, the states which are not in F are represented by a single circle.

Example:



The machine changes the state from q_0 to q_f .

each edge is labelled with (a, X, R)

where, a is the current tape symbol which is replaced by X .

R indicates the read write head is involved towards right by one position.

L - indicates that the read-write head is moved towards left by one position.

Language acceptability by Turing Machines

Let us consider the TM, $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$. A string w in Σ^* is said to be accepted by M if $q_0 w \vdash^* \alpha_1 p \alpha_2$ for some $p \in F$ & $\alpha_1, \alpha_2 \in \Sigma^*$.

M does not accept w if the machine M either halts in a non-accepting state or does not halt.

Design of Turing Machines.

The basic guidelines for designing a TM are:

i). After scanning the symbol below the read/write head is to know what activity has to be done in future. the machine must remember all the scanned symbols and the symbols to be scanned the machine can remember the scanned symbol by moving on to next state.

(ii) The number of states must be minimized. This can be achieved by changing the states only when there is a change in the written ~~the~~ symbol or when there is a change in the movement of the Read/Write head.

Types of Turing Machine:

- i) Multi-dimensional Turing Machine.
- ii) Multi-tape Turing Machine
- iii) Non-deterministic Turing Machine.

i) Multi-Dimensional Turing Machine.

Turing machine has multi-dimensional tape, with a read-write head. It uses basic concept of Turing machine with modification of transition function as shown in figure.

$$\delta: (Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D\})$$

where, U & D Specifies movement of the read-write head up and down.

	a	b			
	b	B			
	B	a			

	a	X			
	b	Y			
	B	Z			

FC
q₀

$$(q_0, b, B, a) = q_1, X, Y, Z$$

FC
q₁

figure: Multi-dimensional Turing Machine.

ii) Multi-tape Turing Machine.

Turing Machine has multi tape, means standard TM with more number of tapes.

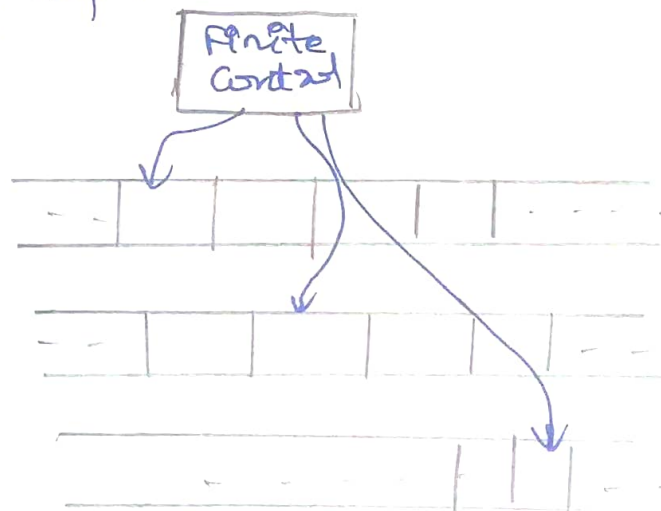


figure: Multitape Turing Machine.

The Component of Multitape Turing Machine are:

* Finite Control * Multiple Read-write heads.

- Each tape divided into cells which can hold any symbol from the given alphabet to start with TM should be in start state q_0 .
- If the read/write head is pointing to tape moves towards right the R/W head pointing to tape 2 & tape 3 may move towards right or left depending on the transition.
- The move of the multitape TM depends on the current state of scanned symbol by each of the tape heads.

(iii) Non-deterministic Turing Machine.

The non-deterministic Turing Machine is a 7-tuple.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F).$$

It is clear from the definition of δ that for each state q and tape symbol x , $\delta(q, x)$ is a set of triples.

$$\{(q_1, x_1, D), (q_2, x_2, D), (q_3, x_3, D), (q_4, x_4, D)\}$$

Where,

* $i \rightarrow$ finite integer

* $D \rightarrow$ direction with left or right.

* The machine can choose any of the triples as the next move.

Halting Problem.

UNDECIDABILITY: A language that is not Recursively Enumerable

Undecidability refers to the inherent impossibility of solving certain problems algorithmically for all possible inputs. In the context of Turing machines and formal languages, a problem is undecidable if there is no Turing machine that can always halt and correctly determine whether a given input belongs to a specific language.

A language that is not Recursively Enumerable

If a language is not recursively enumerable, it means there's no Turing machine that can even list out all the strings in the language, let alone decide membership for all inputs. This signifies a higher level of undecidability compared to RE languages, where at least enumeration is possible.

Key points to Remember

- * Every recursive language is recursively enumerable, but not vice versa.
- * The halting problem is a classic example of a language that is recursively enumerable but not recursive. It's possible to list out (enumerate) Turing machine and input pairs that halt, but it's impossible to design a Turing Machine that always

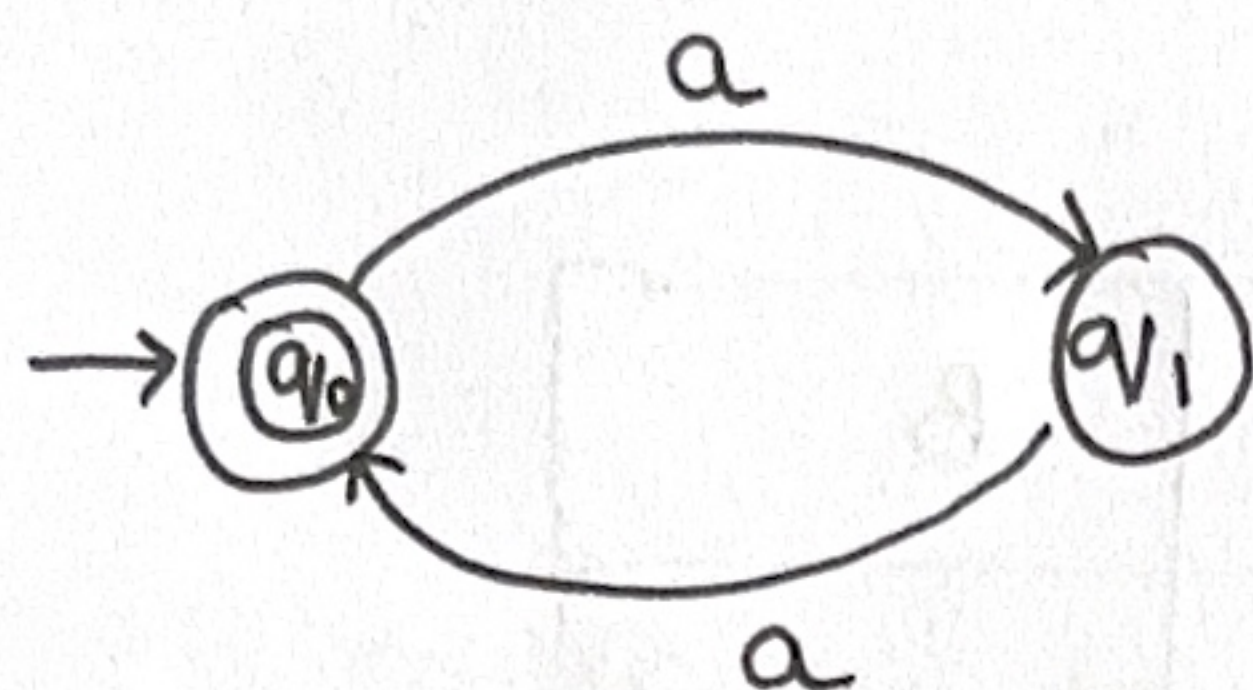
halts and correctly decides whether any given Turing machine will halt on a specific input.

The concept of undecidability is fundamental in computer science, as it sets limits on what can be computed algorithmically. Understanding the distinction between recursive, recursively enumerable, and non-RE languages helps us grasp the complexities and boundaries of computation.

Problems

- ① Design a TM to accept even number of a's. Show the sequence of moves made by TM for the string aa and aaa.

Soln:- DFA



$$\delta(q_0, B) = (q_f, B, R)$$

$$\delta(q_0, a) = (q_1, B, R)$$

Transition table

δ	Tape symbol (τ)	
	a	B
q_0	(q_1, B, R)	(q_f, B, R)
q_1	(q_0, B, R)	-
q_f	-	-

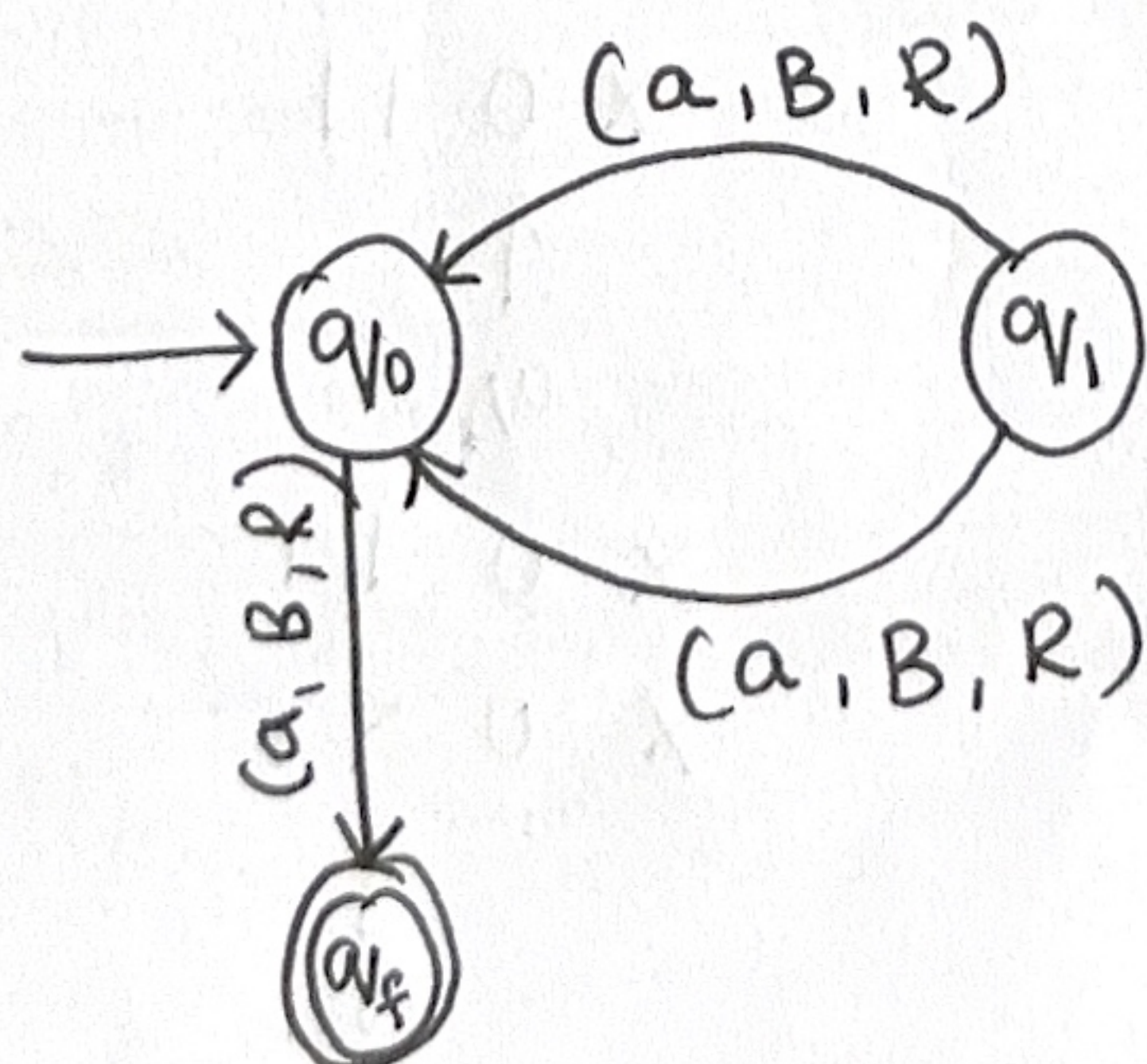
Moves for string aa:

$q_0 \text{ aaB} \xrightarrow{\quad} Bq_1 \text{ aB} \xrightarrow{\quad} q_f \text{ BR (accept)}$

Moves for string aaa:

$q_0 \text{ aaaB} \xrightarrow{\quad} Bq_1 \text{ aaB} \xrightarrow{\quad} BBq_0 \text{ aB} \xrightarrow{\quad} BBBq_1 \text{ B (reject)}$

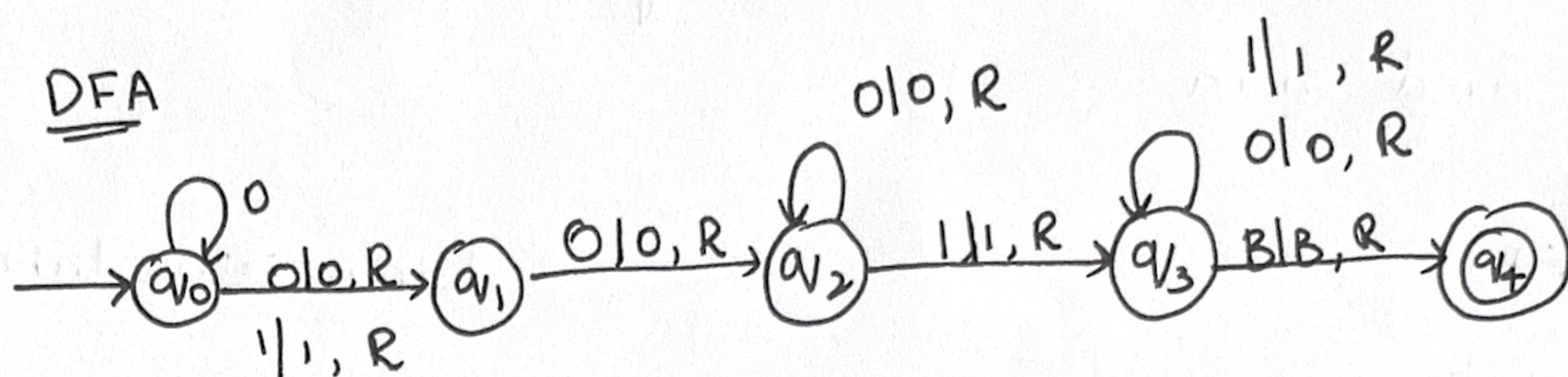
Transition diagram



② Design a TM to accept the following languages

$L = \{w \mid w \in (0+1)^*\}$ containing the substring 001

Soln :-



State	0	1	B
q_0	$q_1, 0, R$	$q_0, 1, R$	
q_1	$q_2, 0, R$	$q_0, 1, R$	
q_2	$q_2, 0, R$	$q_3, 1, R$	
q_3	$q_3, 0, R$	$q_3, 1, R$	q_4, B, R
q_4	-	-	-

③ Design a TM to accept the following language

$L = \{0^n 1^n \mid n \geq 1\}$

$$\delta(q_0, 0) = (q_1, x, R)$$

0 0 1 1

$$\delta(q_1, 1) = (q_1, 0, R)$$

x 0 1 1

$$\delta(q_1, 1) = (q_2, Y, L)$$

↑

q_1

x 0 1 1

$$\delta(q_2, Y) = (q_2, Y, L)$$

x 0 Y 1

$$\delta(q_2, 0) = (q_2, 0, L)$$

↑

q_2

x 0 Y 1

$$\delta(q_2, x) = (q_0, x, R)$$

$$\delta(q_1, y) = (q_1, y, R)$$

$$\delta(q_2, x) = (q_0, x, R)$$

$$\delta(q_0, y) = (q_3, y, R)$$

$$\delta(q_3, y) = (q_3, y, R)$$

$$\delta(q_3, B) = (q_4, B, R)$$

$x \ 0 \ y \ 1$
 $\uparrow \ \uparrow$
 $q_2 \ q_2$

$x \ 0 \ y \ 1$
 $\uparrow$
 q_0

$x \ x \ y \ 1$
 $\uparrow \ \uparrow$
 $q_1 \ q_1$

$x \ x \ y \ y$
 $\uparrow$
 q_2

Turing machine to accept the language $xxyx$

$L = \{0^n 1^n \mid n \geq 1\}$ is given by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where,

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, 1, x, y, B\}$$

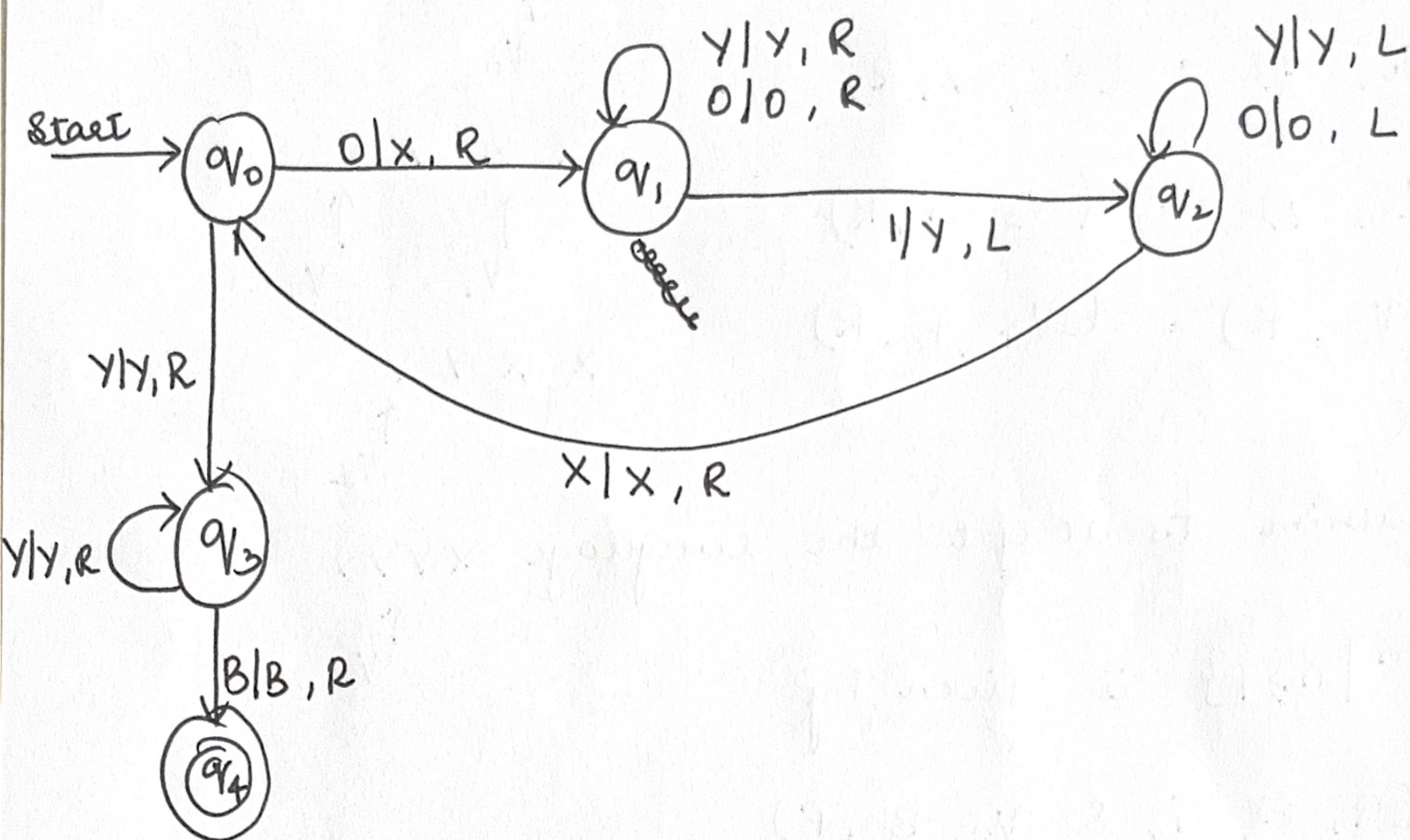
δ is shown in the form of transition table

δ	Tape Symbol (Γ)				
	0	1	x	y	B
q_0	(q_1, x, R)	-	-	(q_3, y, R)	-
q_1	$(q_1, 0, R)$	(q_2, y, L)	-	(q_1, y, R)	-
q_2	$(q_2, 0, L)$	-	(q_0, x, R)	(q_2, y, L)	-
q_3	-	-	-	(q_3, y, R)	(q_4, B, R)
q_4	-	-	-	-	-

q_0 is the start state

B blank character

$F = \{q_4\}$ is the final state



Sequence of moves or computations (IDs) for the string 0011

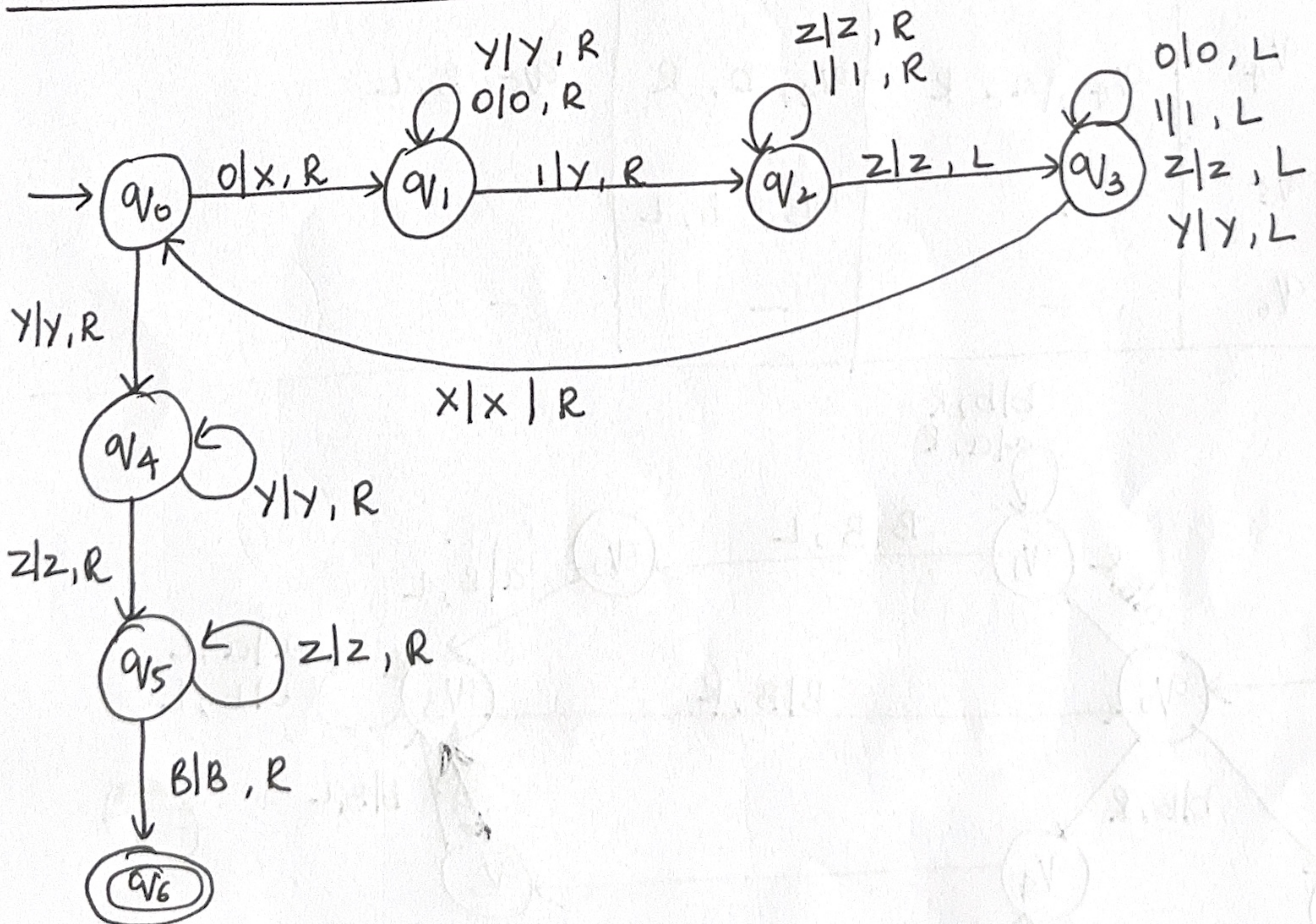
$q_0 0011 \vdash xq_1 011 \vdash x0q_1 11 \vdash xq_2 0Y1 \vdash q_2 x0Y1$
 $\vdash xq_0 0Y1 \vdash xxq_1 Y1 \vdash xxYq_1 \vdash xxq_2 YY$
 $\vdash xq_2 xYY \vdash xxq_0 YY \vdash xxYq_3 Y \vdash xxYYq_3$
 $\vdash xxYYBq_4 \text{ (final ID)}$

④ Design a Turing machine to accept the following language

$$L = \{0^n 1^n 2^n \mid n \geq 1\}$$

Solⁿ

State	T						
	0	1	2	Z	Y	X	B
q_0	q_1, X, R				q_4, Y, R		
q_1	$q_1, 0, R$	q_2, Y, R			q_1, Y, R		
q_2		$q_2, 1, R$	q_3, Z, L	q_2, Z, R			
q_3	$q_3, 0, L$	$q_3, 1, L$		q_3, Z, L	q_3, Y, L	q_0, X, R	
q_4				q_5, Z, R	q_4, Y, R		
q_5				q_5, Z, R			q_0, B, R



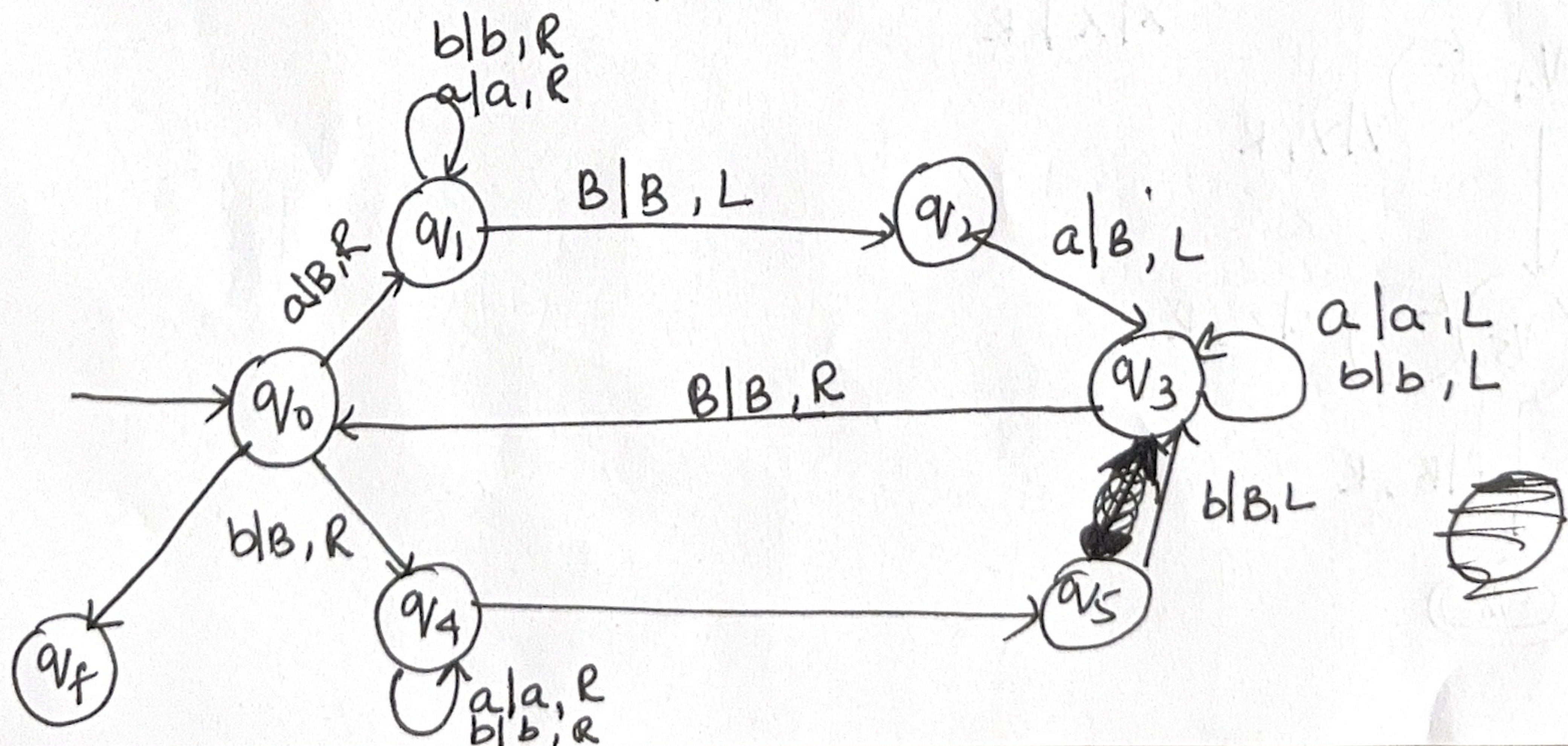
⑤ Design a TM to accept the language
 $L = \{ww^R \mid w \in \{a, b\}^* \mid n \geq 0\}$

Show the sequence of moves made by the TM for the string abba.

Solⁿ

$L = \{ww^R \mid w \in \{a, b\}^* \mid n \geq 0\}$

	a	b	B
q_0	q_1, B, R	q_4, B, R	q_7, B, R
q_1	q_1, a, R	q_1, b, R	q_2, B, L
q_2	q_3, B, L		
q_3	q_3, a, L	q_3, b, L	q_0, B, R
q_4	q_4, a, R	q_4, b, R	q_5, B, L
q_5		q_3, B, L	
q_6	—	—	—



Sequence of moves made for the string abba

$q_0 a b b a B \vdash Bq_1, b b a B \vdash Bbq_1, b a B$

$\vdash Bbbq_1, a B \vdash Bbbaq_1, B \vdash Bbbaq_2, B$

$\vdash Bbbq_3 BB \vdash Bbq_3 b BB \vdash Bq_3 bb BB$

$\vdash Bq_0 bb BB \vdash BBq_4 b BB \vdash BBbq_4 BB$

$\vdash BBbq_5 BB \vdash BBq_3 BBB \vdash BB Bq_0 BB$

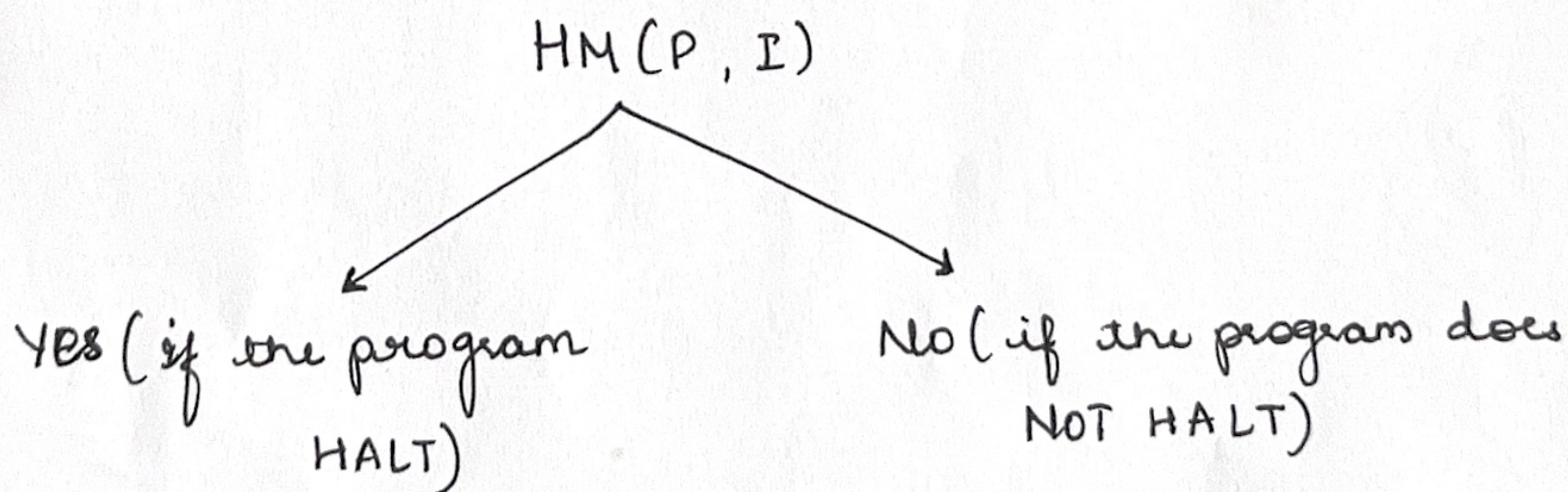
$\vdash BBBBq_f \text{ (accept)}$

The Halting problem

The halting problem is a fundamental problem in computer science that explores the limits of what computers can decide. It asks whether, given a computer program and an input, it is possible to determine in advance whether the program will eventually halt (stop running) or run forever on the input.

The halting problem is undecidable, meaning no algorithm exists that can solve it for all possible programs & inputs. This undecidability is proven using a proof by contradiction:

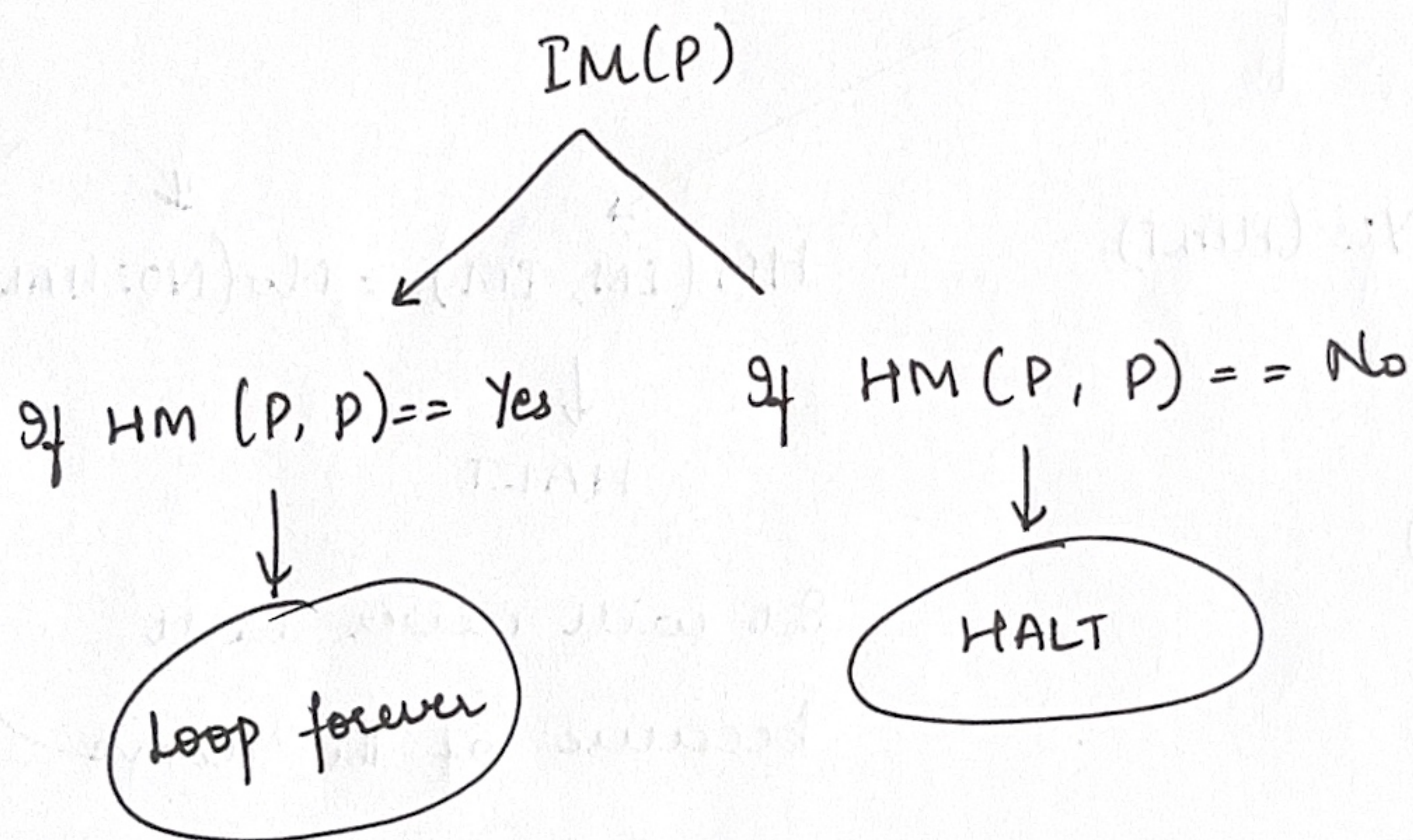
- ① Assume a Halting machine (HM) exists that can determine whether any program P halts on input I . $HM(P, I)$ would output YES if P halts on I and No if it runs forever.



② construct an inverted Halting machine (HM) that takes a program P as input.

IM behaves as follows:-

- If $HM(P, P)$ returns YES (meaning P halts on input P), then IM loops forever.
- If $HM(P, P)$ returns No (meaning P runs forever on input P), then IM halts.



③ consider what happens when IM is given itself as input ($IM(IM)$).

- If $IM(IM)$ halts, then according to IM's definition, $HM(IM, IM)$ must have returned No (meaning IM runs forever on input IM). This is a contradiction.
- If $IM(IM)$ runs forever, then $HM(IM, IM)$ must have returned YES (meaning IM halts on input IM). This is also a contradiction.

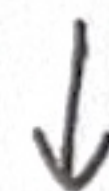
IM (IM)

HM (IM, IM) == Yes (HALT)



Loop forever

HM (IM, IM) == No (NOT HALT)



HALT

It will never halt
because of the above
not halting condition

This contradiction arises because we assumed the existence of a Halting Machine (HM). Therefore, such a machine cannot exist, and the halting problem is undecidable.

The halting problem is an example of a recursively enumerable language that is not recursive.